

AI-DND

TECHNICAL_ARCHITECTURE.md

Technical Architecture Document

Version 0.1

1. PURPOSE

This document defines the technical architecture of AI-DND.

Codex must use this document as the main technical reference before writing or changing code.

The goal is to build a scalable browser RPG with:

- frontend prototype first;
 - modular game systems;
 - AI-ready architecture;
 - persistent world state;
 - future backend support;
 - future local LLM support.
-

2. DEVELOPMENT STRATEGY

The project must be developed in phases.

Phase 1 — Frontend Prototype

No backend.

No database.

No AI server.

Goal:

- main menu;
- character creation;

- world map;
 - travel;
 - event scene;
 - dialogue input;
 - thought panel;
 - basic UI.
-

Phase 2 — Game Rules

Add local game systems:

- character stats;
 - inventory;
 - D20;
 - combat;
 - injuries;
 - travel events;
 - journal.
-

Phase 3 — Data Driven World

Move content into data files:

- NPCs;
 - locations;
 - items;
 - skills;
 - factions;
 - quests;
 - events.
-

Phase 4 — Backend

Add backend for:

- saves;
 - world state;
 - simulation;
 - AI requests;
 - persistence.
-

Phase 5 — AI Integration

Add local AI systems:

- NPC AI;
 - AI Dungeon Master;
 - Player Intent Parser;
 - AI Narrator;
 - memory retrieval.
-

3. CORE TECHNOLOGY STACK

Frontend

Use:

- TypeScript;
- React;
- Vite;
- PixiJS;
- CSS Modules or plain CSS;
- Zustand or lightweight state manager.

Frontend responsibility:

- render screens;
 - display UI;
 - receive player input;
 - show world map;
 - show event scenes;
 - play animations;
 - call game systems.
-

Backend

Use later:

- Node.js;
- Express or Fastify;
- TypeScript;
- PostgreSQL;
- Redis optional.

Backend responsibility:

- save world state;
 - simulate world;
 - store NPC memory;
 - process AI requests;
 - manage long-term data.
-

AI Server

Use later:

- LM Studio;
- OpenAI-compatible local API;
- local model only.

AI server responsibility:

- generate NPC dialogue;
- generate narration;
- assist with intent parsing;
- create scene descriptions.

AI must never decide dice results.

4. PROJECT STRUCTURE

Recommended root structure:

```
AI-DND/  
├─ client/  
├─ server/  
├─ shared/  
├─ data/  
├─ docs/  
├─ assets/  
├─ tools/  
└─ README.md
```

5. CLIENT STRUCTURE

```
client/
├─ public/
├─ src/
│   ├─ app/
│   ├─ assets/
│   ├─ components/
│   ├─ data/
│   ├─ engine/
│   ├─ features/
│   ├─ game/
│   ├─ scenes/
│   ├─ state/
│   ├─ styles/
│   ├─ systems/
│   ├─ types/
│   └─ main.tsx
├─ package.json
└─ vite.config.ts
```

6. FRONTEND FOLDER RESPONSIBILITIES

app/

Application bootstrap.

Contains:

- App.tsx;
- routing;
- layout;
- app initialization.

components/

Reusable UI components.

Examples:

- Button;
 - Panel;
 - Modal;
 - InputBox;
 - CharacterThoughts;
 - DialogueWindow.
-

engine/

Low-level game engine systems.

Examples:

- GameLoop;
 - EventBus;
 - AssetLoader;
 - SceneManager;
 - TimeManager.
-

scenes/

Visual game screens.

Examples:

- MainMenuScene;
 - CharacterCreationScene;
 - WorldMapScene;
 - EventScene;
 - CombatScene;
 - StoneSleepScene.
-

systems/

Pure gameplay systems.

Examples:

- DiceSystem;
- CombatSystem;

- InventorySystem;
 - ThoughtSystem;
 - IntentSystem;
 - TravelSystem;
 - InjurySystem.
-

state/

Global frontend state.

Examples:

- playerStore;
 - worldStore;
 - uiStore;
 - dialogueStore.
-

data/

Temporary frontend data.

Used before backend exists.

Examples:

- items.json;
 - npcs.json;
 - locations.json;
 - factions.json;
 - skills.json.
-

types/

Shared TypeScript interfaces.

Examples:

- Player;
- NPC;
- Item;
- Skill;
- WorldEvent;

- Memory;
 - Location.
-

7. SERVER STRUCTURE

```
server/  
├─ src/  
│   ├── api/  
│   ├── ai/  
│   ├── database/  
│   ├── simulation/  
│   ├── systems/  
│   ├── world/  
│   ├── saves/  
│   └─ index.ts  
├─ package.json  
└─ tsconfig.json
```

8. SHARED STRUCTURE

```
shared/  
├─ types/  
├─ constants/  
├─ schemas/  
└─ utils/
```

Shared folder contains code used by both client and server.

No UI code is allowed in shared.

9. DATA STRUCTURE

```
data/  
├─ factions/  
├─ items/  
├─ locations/  
└─ lore/
```



```
|— monsters/  
|— npcs/  
|— quests/  
|— skills/  
|— spells/  
└— world/
```

All content must be data-driven when possible.

Codex must not hardcode game content into logic files unless explicitly requested.

10. ARCHITECTURAL PRINCIPLES

Principle 1 — Data Driven Design

Game content lives in data files.

Logic files process data.

Principle 2 — One System, One Responsibility

Each system must have one clear purpose.

Bad:

CombatSystem also manages inventory.

Good:

CombatSystem calculates combat. InventorySystem manages items.

Principle 3 — AI Does Not Own Rules

AI systems may describe, interpret or suggest.

AI systems may not decide:

- hit chance;
- dice result;

- damage;
 - death;
 - item creation;
 - quest success.
-

Principle 4 — Game Engine Is Source of Truth

All real outcomes come from game systems.

Principle 5 — UI Is Not Logic

React components must not contain major game rules.

UI calls systems.

Systems return results.

UI displays results.

11. GAME SYSTEM PIPELINE

Player Input ↓ IntentSystem ↓ RuleValidator ↓ Relevant Game System ↓ DiceSystem ↓ Result Object ↓
AI Narrator ↓ UI Display

12. PLAYER INTENT ARCHITECTURE

IntentSystem receives free text.

It outputs structured intent.

Example:

Input:

"I hit his left arm with my sword."

Output:

```
{
  "type": "combat.attack",
  "weapon": "sword",
  "target": "left_arm",
  "tone": "aggressive"
}
```

Then RuleValidator checks if action is possible.

13. RULE VALIDATOR

RuleValidator checks:

- equipment;
- skill;
- training;
- injuries;
- fatigue;
- context;
- target availability.

RuleValidator never directly narrates.

It returns structured result.

Example:

```
{
  "allowed": false,
  "reason": "no_sword_equipped",
  "fallbackAction": "reach_for_missing_weapon"
}
```

14. DICE SYSTEM

DiceSystem is deterministic from rules.

Responsibilities:

- roll d20;

- roll damage dice;
- apply modifiers;
- detect critical success;
- detect critical failure.

DiceSystem must be testable.

15. COMBAT SYSTEM

CombatSystem handles:

- initiative;
- attack validation;
- defense;
- damage;
- body part targeting;
- wounds;
- bleeding;
- fractures;
- surrender;
- escape.

CombatSystem returns structured combat results.

16. THOUGHT SYSTEM

ThoughtSystem generates character thoughts based on:

- player knowledge;
- skills;
- situation;
- fear;
- injury;
- fatigue;
- environment.

Thoughts are suggestions, not commands.

17. WORLD MAP SYSTEM

WorldMapSystem handles:

- map locations;
 - travel routes;
 - travel time;
 - random events;
 - route danger;
 - discovery.
-

18. EVENT SCENE SYSTEM

EventSceneSystem handles:

- current encounter;
 - active NPCs;
 - scene description;
 - available context;
 - transition to combat or dialogue.
-

19. SAVE SYSTEM

During Phase 1, saves may use:

- localStorage.

Later backend save system uses:

- PostgreSQL.

Save data must include:

- player;
 - world state;
 - NPC memory;
 - rumors;
 - history;
 - Stone Sleep data.
-

20. STONE SLEEP SYSTEM

StoneSleepSystem handles:

- player logout;
 - offline time calculation;
 - time compression;
 - world simulation summary;
 - awakening screen.
-

21. WORLD SIMULATION SYSTEM

WorldSimulationSystem handles:

- economy;
- faction changes;
- deaths;
- diseases;
- rumors;
- wars;
- NPC changes.

In frontend prototype, use simplified mock simulation.

22. AI MODULES

AI must be modular.

Required AI modules:

- AI_NPC;
 - AI_DungeonMaster;
 - AI_Narrator;
 - AI_IntentAssistant;
 - AI_LoreKeeper.
-

23. AI PIPELINE

Game System Result ↓ Context Builder ↓ Relevant Memory Retrieval ↓ Lore Filter ↓ AI Prompt ↓ AI Response ↓ Safety / Rule Check ↓ UI Display

24. LORE KEEPER

LoreKeeper ensures:

- no modern references;
- no breaking world rules;
- no impossible facts;
- no hidden knowledge exposure.

25. TESTING STRATEGY

Use automated tests for pure systems:

- DiceSystem;
- CombatSystem;
- InventorySystem;
- RuleValidator;
- IntentSystem;
- TravelSystem.

UI may be tested later.

26. DEVELOPMENT ORDER

Codex should implement in this order:

1. Create React + Vite frontend.
2. Build main menu.
3. Build character creation.
4. Build world map screen.
5. Build event scene screen.
6. Build dialogue input.
7. Build thought panel.
8. Build local data files.
9. Build DiceSystem.
10. Build IntentSystem mock.
11. Build RuleValidator.
12. Build TravelSystem.
13. Build CombatSystem.
14. Build InventorySystem.
15. Build save to localStorage.

16. Add backend later.

27. CODING RULES

Use TypeScript everywhere.

No JavaScript files unless unavoidable.

Use strict typing.

Avoid giant files.

Preferred maximum file length:

300 lines.

If file grows larger:

split it.

28. NAMING CONVENTIONS

Use PascalCase for:

- classes;
- React components;
- types;
- interfaces.

Use camelCase for:

- functions;
- variables;
- methods.

Use kebab-case for:

- asset filenames.
-

29. RESULT OBJECTS

Systems must return structured objects.

Bad:

```
return "You hit the enemy."
```

Good:

```
return {  
  success: true,  
  target: "torso",  
  damage: 8,  
  effects: ["bleeding_light"]  
}
```

Narration happens separately.

30. ERROR HANDLING

Game systems should not crash the app.

They should return failure results.

Example:

```
{  
  success: false,  
  reason: "missing_weapon"  
}
```

31. ASSET STRATEGY

Assets are stored in:

```
client/public/assets/
```

Recommended categories:

```
assets/  
├─ backgrounds/  
├─ ui/  
├─ icons/  
├─ maps/  
├─ characters/  
└─ audio/
```

32. FRONTEND PROTOTYPE RULE

In Phase 1, use mock data.

Do not block progress waiting for backend.

33. BACKEND RULE

Backend must not be added until frontend prototype is playable.

Playable prototype means:

- menu works;
 - character creation works;
 - map works;
 - travel works;
 - encounter screen works;
 - text input works.
-

34. AI RULE

AI integration must not be added until:

- IntentSystem exists;
- RuleValidator exists;
- DiceSystem exists;

- basic EventScene exists.
-

35. CORE TECHNICAL RULE

The game must remain playable even without AI.

AI improves narration and NPC behavior.

AI must not be required for basic game logic.

36. FINAL ARCHITECTURE GOAL

The final project should support:

- persistent living world;
 - AI NPC dialogue;
 - dynamic events;
 - world simulation;
 - Stone Sleep;
 - combat;
 - injuries;
 - reputation;
 - rumors;
 - local LLM integration.
-

37. CORE RULE

Codex writes code.

The documents define the game.

If code conflicts with documentation, the documentation wins.